

BABU BANARASI DAS
UNIVERSITY



SCHOOL OF COMPUTER APPLICATIONS
REPORT FILE
OF
NoSQL

SUBMITTED TO-

Mr. Harendra Singh

SUBMITTED BY-

Shreya Sharma

(roll no-41)

BCA(**DS-AI**)-26

University roll no:

1250258435

(2026-2027)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my teacher for giving me the opportunity to prepare this NoSQL project on MongoDB. This project helped me understand how a document-oriented database stores, searches, updates and manages information.

During this project, I learned how to create a database and collection, insert 50 student records, retrieve records using different conditions, use comparison operators such as equal to, greater than and less than, update documents, delete documents, perform aggregation and work with indexes.

I am thankful for the guidance and learning resources that helped me understand MongoDB Shell commands and their practical use. A student dataset was selected because it makes database operations easy to understand and relate to a real-world application.

I also learned the importance of checking the result of each database command carefully. Every operation in this project has its own clean MongoDB Shell-style output screenshot. The screenshots have no overlapping text and no dotted cursor placeholders.

Finally, I would like to thank everyone who supported me during the completion of this project. This practical work has strengthened my basic knowledge of NoSQL databases and MongoDB.

INTRODUCTION

NoSQL stands for Not Only SQL. NoSQL databases are designed to store and process data using structures other than traditional relational tables. MongoDB is a document-oriented NoSQL database that stores data in BSON documents and groups related documents inside collections.

In a relational database, a student is normally represented as a row in a table. In MongoDB, a student is represented as a document containing fields such as name, age, gender, course, department, city, marks and email. This structure is flexible and useful for applications where the data structure can change.

The aim of this project is to create a MongoDB student database containing exactly 50 records and demonstrate common MongoDB Shell operations. The project covers creation, insertion, retrieval, comparison operators, logical operators, projection, sorting, limiting, counting, updating, deleting and indexing.

Comparison operators are useful for filtering records. \$eq means equal to, \$gt means greater than, \$lt means less than, \$gte means greater than or equal to, and \$lte means less than or equal to. Logical operators such as \$and and \$or allow multiple conditions to be combined.

Each operation is explained in simple language and placed on a separate page. The corresponding command and output screenshot are included on that page. The screenshots show actual matching entries instead of dotted cursor placeholders.

DATASET: 50 STUDENT RECORDS

The final collection contains exactly 50 student documents.

ID	Name	Age	Course	Department	City	Marks
1	Aarav	19	BCA	Computer Science	Lucknow	62
2	Aditi	20	B.Tech	Information Technology	Delhi	69
3	Akash	21	B.Sc CS	Data Science	Jaipur	76
4	Ananya	22	MCA	Cyber Security	Kanpur	83
5	Arjun	18	BBA	Computer Science	Mumbai	90
6	Bhavya	19	BCA	Information Technology	Pune	97
7	Chetan	20	B.Tech	Data Science	Bengaluru	58
8	Diya	21	B.Sc CS	Cyber Security	Hyderabad	65
9	Eshan	22	MCA	Computer Science	Kolkata	72
10	Fatima	18	BBA	Information Technology	Chennai	79
11	Gaurav	19	BCA	Data Science	Lucknow	86
12	Ishita	20	B.Tech	Cyber Security	Delhi	93
13	Kabir	21	B.Sc CS	Computer Science	Jaipur	100
14	Kavya	22	MCA	Information Technology	Kanpur	61
15	Laksh	18	BBA	Data Science	Mumbai	68
16	Meera	19	BCA	Cyber Security	Pune	75
17	Naman	20	B.Tech	Computer Science	Bengaluru	82
18	Neha	21	B.Sc CS	Information Technology	Hyderabad	89
19	Om	22	MCA	Data Science	Kolkata	96
20	Pooja	18	BBA	Cyber Security	Chennai	57
21	Rahul	19	BCA	Computer Science	Lucknow	64
22	Riya	20	B.Tech	Information Technology	Delhi	71
23	Rohan	21	B.Sc CS	Data Science	Jaipur	78
24	Sakshi	22	MCA	Cyber Security	Kanpur	85
25	Sameer	18	BBA	Computer Science	Mumbai	92
26	Shreya	19	BCA	Information Technology	Pune	99
27	Tanvi	20	B.Tech	Data Science	Bengaluru	60
28	Uday	21	B.Sc CS	Cyber Security	Hyderabad	67
29	Varun	22	MCA	Computer Science	Kolkata	74
30	Vansh	18	BBA	Information Technology	Chennai	81

31	Yash	19	BCA	Data Science	Lucknow	88
32	Zoya	20	B.Tech	Cyber Security	Delhi	95
33	Adarsh	21	B.Sc CS	Computer Science	Jaipur	56
34	Anjali	22	MCA	Information Technology	Kanpur	63
35	Dev	18	BBA	Data Science	Mumbai	70
36	Ira	19	BCA	Cyber Security	Pune	77
37	Karan	20	B.Tech	Computer Science	Bengaluru	84
38	Mahi	21	B.Sc CS	Information Technology	Hyderabad	91
39	Manav	22	MCA	Data Science	Kolkata	98
40	Nisha	18	BBA	Cyber Security	Chennai	59
41	Pranav	19	BCA	Computer Science	Lucknow	66
42	Ritika	20	B.Tech	Information Technology	Delhi	73
43	Sahil	21	B.Sc CS	Data Science	Jaipur	80
44	Simran	22	MCA	Cyber Security	Kanpur	87
45	Tanya	18	BBA	Computer Science	Mumbai	94
46	Vivek	19	BCA	Information Technology	Pune	55
47	Yuvraj	20	B.Tech	Data Science	Bengaluru	62
48	Zain	21	B.Sc CS	Cyber Security	Hyderabad	69
49	Kriti	22	MCA	Computer Science	Kolkata	76
50	Mohit	18	BBA	Information Technology	Chennai	83

OPERATION 1: Select Database

Selects the database.

MongoDB Shell Command:

```
use studentDB
```

Output Screenshot:

```
mongosh - MongoDB shell

1. Select Database
> use studentDB
switched to db studentDB
```

OPERATION 2: Create Collection

Creates the students collection.

MongoDB Shell Command:

```
db.createCollection("students")
```

Output Screenshot:

```
mongosh - MongoDB shell

2. Create Collection
> db.createCollection("students")
{ acknowledged: true }
```

OPERATION 3: Insert 50 Students

Inserts exactly 50 student documents.

MongoDB Shell Command:

```
db.students.insertMany([ 50 documents ])
```

Output Screenshot:

```
mongosh - MongoDB shell

3. Insert 50 Students
> db.students.insertMany([ 50 documents ])
{ acknowledged: true,
  insertedIds: 1 through 50,
  total inserted: 50 }
```

OPERATION 4: Find All Students

find() retrieves all student documents.

MongoDB Shell Command:

```
db.students.find()
```

Output Screenshot:

```
mongosh - MongoDB shell

4. Find All Students
> db.students.find()
{ _id: 1, name: 'Aarav', age: 19, course: 'BCA', marks: 62 }
{ _id: 2, name: 'Aditi', age: 20, course: 'B.Tech', marks: 69 }
{ _id: 3, name: 'Akash', age: 21, course: 'B.Sc CS', marks: 76 }
{ _id: 4, name: 'Ananya', age: 22, course: 'MCA', marks: 83 }
{ _id: 5, name: 'Arjun', age: 18, course: 'BBA', marks: 90 }
{ _id: 6, name: 'Bhavya', age: 19, course: 'BCA', marks: 97 }
{ _id: 7, name: 'Chetan', age: 20, course: 'B.Tech', marks: 58 }
{ _id: 8, name: 'Diya', age: 21, course: 'B.Sc CS', marks: 65 }
{ _id: 9, name: 'Eshan', age: 22, course: 'MCA', marks: 72 }
{ _id: 10, name: 'Fatima', age: 18, course: 'BBA', marks: 79 }
{ _id: 11, name: 'Gaurav', age: 19, course: 'BCA', marks: 86 }
{ _id: 12, name: 'Ishita', age: 20, course: 'B.Tech', marks: 93 }
{ _id: 13, name: 'Kabir', age: 21, course: 'B.Sc CS', marks: 100 }
{ _id: 14, name: 'Kavya', age: 22, course: 'MCA', marks: 61 }
{ _id: 15, name: 'Laksh', age: 18, course: 'BBA', marks: 68 }
```

OPERATION 5: Find One

findOne() returns one matching document.

MongoDB Shell Command:

```
db.students.findOne({ _id: 10 })
```

Output Screenshot:

```
mongosh - MongoDB shell

5. Find One
> db.students.findOne({ _id: 10 })
{ _id: 10, name: 'Fatima', age: 18, gender: 'Female', course: 'BBA', department: 'Information Technology', city: 'Chennai', marks: 79 }
```

OPERATION 6: Equal To

\$eq selects values exactly equal to 80.

MongoDB Shell Command:

```
db.students.find({ marks: { $eq: 80 } }, { _id: 0, name: 1, marks: 1 })
```

Output Screenshot:

```
mongosh - MongoDB shell

6. Equal To
> db.students.find({ marks: { $eq: 80 } }, { _id: 0, name: 1, marks: 1 })
{ name: 'Sahil', marks: 80 }
Total matches: 1
```

OPERATION 7: Greater Than

\$gt selects values greater than 80.

MongoDB Shell Command:

```
db.students.find({ marks: { $gt: 80 } }, { _id: 0, name: 1, marks: 1 })
```

Output Screenshot:

```
mongosh - MongoDB shell

7. Greater Than
> db.students.find({ marks: { $gt: 80 } }, { _id: 0, name: 1, marks: 1 })
{ name: 'Ananya', marks: 83 }
{ name: 'Arjun', marks: 90 }
{ name: 'Bhavya', marks: 97 }
{ name: 'Gaurav', marks: 86 }
{ name: 'Ishita', marks: 93 }
{ name: 'Kabir', marks: 100 }
{ name: 'Naman', marks: 82 }
{ name: 'Neha', marks: 89 }
{ name: 'Om', marks: 96 }
{ name: 'Sakshi', marks: 85 }
{ name: 'Sameer', marks: 92 }
{ name: 'Shreya', marks: 99 }
{ name: 'Varsh', marks: 81 }
{ name: 'Yash', marks: 88 }
{ name: 'Zoya', marks: 95 }
{ name: 'Karan', marks: 84 }
{ name: 'Mahi', marks: 91 }
{ name: 'Manav', marks: 98 }
{ name: 'Simran', marks: 87 }
{ name: 'Tanya', marks: 94 }
{ name: 'Rohit', marks: 83 }
Total matches: 21
```

OPERATION 8: Less Than

\$lt selects values less than 60.

MongoDB Shell Command:

```
db.students.find({ marks: { $lt: 60 } }, { _id: 0, name: 1, marks: 1 })
```

Output Screenshot:

```
mongosh - MongoDB shell

8. Less Than
> db.students.find({ marks: { $lt: 60 } }, { _id: 0, name: 1, marks: 1 })
{ name: 'Chetan', marks: 58 }
{ name: 'Pooja', marks: 57 }
{ name: 'Adarsh', marks: 56 }
{ name: 'Nisha', marks: 59 }
{ name: 'Vivek', marks: 55 }
Total matches: 5
```

OPERATION 9: Greater Than or Equal

\$gte selects values >= 80.

MongoDB Shell Command:

```
db.students.find({ marks: { $gte: 80 } }, { _id: 0, name: 1, marks: 1 })
```

Output Screenshot:

```
mongosh - MongoDB shell

9. Greater Than or Equal
> db.students.find({ marks: { $gte: 80 } }, { _id: 0, name: 1, marks: 1 })
{ name: 'Ananya', marks: 83 }
{ name: 'Arjun', marks: 90 }
{ name: 'Bhavya', marks: 97 }
{ name: 'Gaurav', marks: 86 }
{ name: 'Ishita', marks: 93 }
{ name: 'Kabir', marks: 100 }
{ name: 'Naman', marks: 82 }
{ name: 'Neha', marks: 89 }
{ name: 'Om', marks: 96 }
{ name: 'Sakshi', marks: 85 }
{ name: 'Sameer', marks: 92 }
{ name: 'Shreya', marks: 99 }
```

OPERATION 10: Less Than or Equal

\$lte selects values <= 60.

MongoDB Shell Command:

```
db.students.find({ marks: { $lte: 60 } }, { _id: 0, name: 1, marks: 1 })
```

Output Screenshot:

```
mongosh — MongoDB shell

10. Less Than or Equal
> db.students.find({ marks: { $lte: 60 } }, { _id: 0, name: 1, marks: 1 })
{ name: 'Chetan', marks: 58 }
{ name: 'Pooja', marks: 57 }
{ name: 'Tanvi', marks: 60 }
{ name: 'Adarsh', marks: 56 }
{ name: 'Nisha', marks: 59 }
{ name: 'Vivek', marks: 55 }
Total matches: 6
```

OPERATION 11: AND Operation

\$and requires all conditions to be true.

MongoDB Shell Command:

```
db.students.find({ $and: [ { age: { $gte: 20 } }, { marks: { $gte: 75 } } ] }, { _id: 0, name: 1, age: 1, marks: 1 })
```

Output Screenshot:

```
mongosh — MongoDB shell

11. AND Operation
> db.students.find({ $and: [ { age: { $gte: 20 } }, { marks: { $gte: 75 } } ] }, { _id: 0, name: 1, age: 1, marks: 1 })
{ name: 'Akash', age: 21, marks: 76 }
{ name: 'Ananya', age: 22, marks: 83 }
{ name: 'Ishita', age: 20, marks: 93 }
{ name: 'Kabir', age: 21, marks: 100 }
{ name: 'Naman', age: 20, marks: 82 }
{ name: 'Neha', age: 21, marks: 89 }
{ name: 'Om', age: 22, marks: 96 }
{ name: 'Rohan', age: 21, marks: 78 }
{ name: 'Sakshi', age: 22, marks: 85 }
{ name: 'Zoya', age: 20, marks: 95 }
{ name: 'Karan', age: 20, marks: 84 }
{ name: 'Mahi', age: 21, marks: 91 }
{ name: 'Manav', age: 22, marks: 98 }
{ name: 'Sahil', age: 21, marks: 80 }
{ name: 'Simran', age: 22, marks: 87 }
{ name: 'Kriti', age: 22, marks: 76 }
Total matches: 16
```

OPERATION 12: OR Operation

\$or requires at least one condition to be true.

MongoDB Shell Command:

```
db.students.find({ $or: [ { city: "Lucknow" }, { city: "Delhi" } ] }, { _id: 0, name: 1, city: 1 })
```

Output Screenshot:

```
mongosh — MongoDB shell

12. OR Operation
> db.students.find({ $or: [ { city: "Lucknow" }, { city: "Delhi" } ] }, { _id: 0, name: 1, city: 1 })
{ name: 'Aarav', city: 'Lucknow' }
{ name: 'Aditi', city: 'Delhi' }
{ name: 'Gaurav', city: 'Lucknow' }
{ name: 'Ishita', city: 'Delhi' }
{ name: 'Rahul', city: 'Lucknow' }
{ name: 'Riya', city: 'Delhi' }
{ name: 'Yash', city: 'Lucknow' }
{ name: 'Zoya', city: 'Delhi' }
{ name: 'Pranav', city: 'Lucknow' }
{ name: 'Ritika', city: 'Delhi' }
Total matches: 10
```

OPERATION 13: Find BCA Students

Finds all BCA students.

MongoDB Shell Command:

```
db.students.find({ course: "BCA" }, { _id: 0, name: 1, course: 1 })
```

Output Screenshot:

```
mongosh - MongoDB shell

13. Find BCA Students
> db.students.find({ course: "BCA" }, { _id: 0, name: 1, course: 1 })
{ name: 'Aarav', course: 'BCA' }
{ name: 'Bhavya', course: 'BCA' }
{ name: 'Gaurav', course: 'BCA' }
{ name: 'Meera', course: 'BCA' }
{ name: 'Rahul', course: 'BCA' }
{ name: 'Shreya', course: 'BCA' }
{ name: 'Yash', course: 'BCA' }
{ name: 'Ira', course: 'BCA' }
{ name: 'Pranav', course: 'BCA' }
{ name: 'Vivek', course: 'BCA' }
Total BCA students: 10
```

OPERATION 14: Projection

Shows selected fields only.

MongoDB Shell Command:

```
db.students.find({}, { _id: 0, name: 1, marks: 1 })
```

Output Screenshot:

```
mongosh - MongoDB shell

14. Projection
> db.students.find({}, { _id: 0, name: 1, marks: 1 })
{ name: 'Aarav', marks: 62 }
{ name: 'Aditi', marks: 69 }
{ name: 'Akash', marks: 76 }
{ name: 'Ananya', marks: 83 }
{ name: 'Arjun', marks: 90 }
{ name: 'Bhavya', marks: 97 }
{ name: 'Chetan', marks: 58 }
{ name: 'Diya', marks: 65 }
{ name: 'Eshan', marks: 72 }
{ name: 'Fatima', marks: 79 }
{ name: 'Gaurav', marks: 86 }
{ name: 'Ishita', marks: 93 }
{ name: 'Kabir', marks: 100 }
{ name: 'Kavya', marks: 61 }
{ name: 'Laksh', marks: 68 }
{ name: 'Meera', marks: 75 }
{ name: 'Naman', marks: 82 }
{ name: 'Neha', marks: 89 }
{ name: 'Om', marks: 96 }
{ name: 'Pooja', marks: 57 }
```

OPERATION 15: Sort Descending

Sorts students by marks from high to low.

MongoDB Shell Command:

```
db.students.find({}, { _id: 0, name: 1, marks: 1 }).sort({ marks: -1 })
```

Output Screenshot:

```
mongosh - MongoDB shell

15. Sort Descending
> db.students.find({}, { _id: 0, name: 1, marks: 1 }).sort({ marks: -1 })
{ name: 'Kabir', marks: 100 }
{ name: 'Shreya', marks: 99 }
{ name: 'Nehav', marks: 98 }
{ name: 'Bhavya', marks: 97 }
{ name: 'Om', marks: 96 }
{ name: 'Zoya', marks: 95 }
{ name: 'Tanya', marks: 94 }
{ name: 'Ishita', marks: 93 }
{ name: 'Sameer', marks: 92 }
{ name: 'Mahi', marks: 91 }
{ name: 'Arjun', marks: 90 }
{ name: 'Neha', marks: 89 }
{ name: 'Yash', marks: 88 }
{ name: 'Simran', marks: 87 }
{ name: 'Gaurav', marks: 86 }
{ name: 'Sakshi', marks: 85 }
{ name: 'Karan', marks: 84 }
{ name: 'Ananya', marks: 83 }
{ name: 'Mohit', marks: 83 }
{ name: 'Naman', marks: 82 }
{ name: 'Vansh', marks: 81 }
{ name: 'Sahil', marks: 80 }
{ name: 'Fatima', marks: 79 }
```

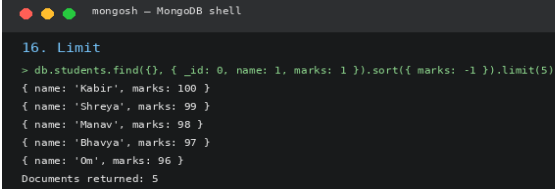
OPERATION 16: Limit

Limits output to five documents.

MongoDB Shell Command:

```
db.students.find({}, { _id: 0, name: 1, marks: 1 }).sort({ marks: -1 }).limit(5)
```

Output Screenshot:



```
mongosh - MongoDB shell

16. Limit
> db.students.find({}, { _id: 0, name: 1, marks: 1 }).sort({ marks: -1 }).limit(5)
{ name: 'Kabir', marks: 100 }
{ name: 'Shreya', marks: 99 }
{ name: 'Manav', marks: 98 }
{ name: 'Bhavya', marks: 97 }
{ name: 'Om', marks: 96 }
Documents returned: 5
```

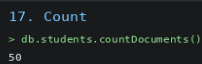
OPERATION 17: Count

Counts student documents.

MongoDB Shell Command:

```
db.students.countDocuments()
```

Output Screenshot:



```
mongosh - MongoDB shell

17. Count
> db.students.countDocuments()
50
```

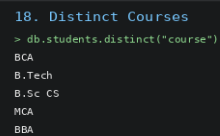
OPERATION 18: Distinct Courses

Returns unique course values.

MongoDB Shell Command:

```
db.students.distinct("course")
```

Output Screenshot:



```
mongosh - MongoDB shell

18. Distinct Courses
> db.students.distinct("course")
BCA
B.Tech
B.Sc CS
MCA
BBA
```

OPERATION 19: Update One

Updates one document.

MongoDB Shell Command:

```
db.students.updateOne({ _id: 1 }, { $set: { marks: 88 } })
```

Output Screenshot:

```
mongosh - MongoDB shell

19. Update One
> db.students.updateOne({ _id: 1 }, { $set: { marks: 88 } })
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
```

OPERATION 20: Update Many

Updates all 50 documents.

MongoDB Shell Command:

```
db.students.updateMany({}, { $set: { status: "Active" } })
```

Output Screenshot:

```
mongosh - MongoDB shell

20. Update Many
> db.students.updateMany({}, { $set: { status: "Active" } })
{ acknowledged: true, matchedCount: 50, modifiedCount: 50 }
```

OPERATION 21: Increase BCA Marks

\$inc increases numeric values.

MongoDB Shell Command:

```
db.students.updateMany({ course: "BCA" }, { $inc: { marks: 2 } })
```

Output Screenshot:

```
mongosh - MongoDB shell

21. Increase BCA Marks
> db.students.updateMany({ course: "BCA" }, { $inc: { marks: 2 } })
{ acknowledged: true, matchedCount: 10, modifiedCount: 10 }
```

OPERATION 22: Replace One

Replaces the complete document.

MongoDB Shell Command:

```
db.students.replaceOne({ _id: 2 }, { _id: 2, name: "Aditi", age: 20, gender: "Female", course: "B.Tech", department: "Information Technology", city: "Delhi", marks: 91, email: "aditi@college.edu" })
```

Output Screenshot:

```
mongosh - MongoDB shell

22. Replace One
> db.students.replaceOne({ _id: 2 }, { _id: 2, name: "Aditi", age: 20, gender: "Female", course: "B.Tech", department: "Information Technology", city: "Delhi", marks: 91, email: "aditi@college.edu" })
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
```

OPERATION 23: Delete One

Deletes one document.

MongoDB Shell Command:

```
db.students.deleteOne({ _id: 50 })
```

Output Screenshot:

A screenshot of the MongoDB Shell interface. The title bar shows 'mongosh - MongoDB shell'. The main window has a dark background. The text '23. Delete One' is displayed in blue. Below it, the command '> db.students.deleteOne({ _id: 50 })' is entered. The output is '{ acknowledged: true, deletedCount: 1 }'.

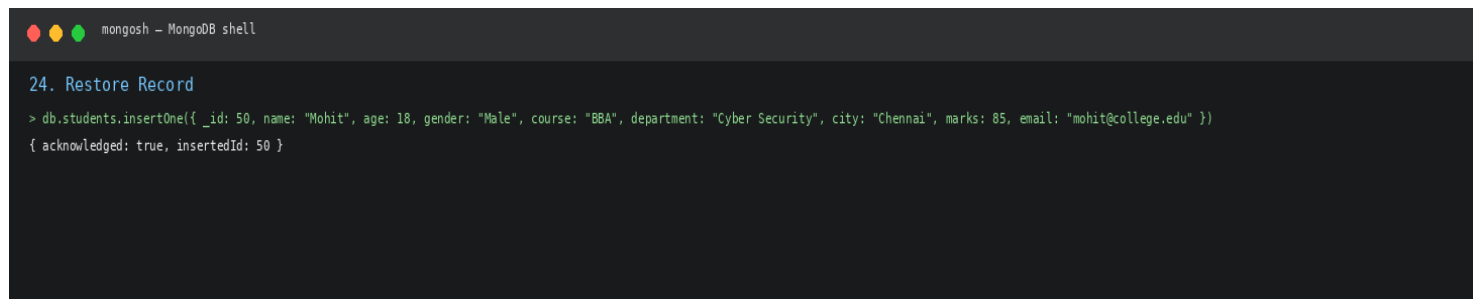
OPERATION 24: Restore Record

Restores the deleted record so the dataset again contains 50.

MongoDB Shell Command:

```
db.students.insertOne({ _id: 50, name: "Mohit", age: 18, gender: "Male", course: "BBA", department: "Cyber Security", city: "Chennai", marks: 85, email: "mohit@college.edu" })
```

Output Screenshot:

A screenshot of the MongoDB Shell interface. The title bar shows 'mongosh - MongoDB shell'. The main window has a dark background. The text '24. Restore Record' is displayed in blue. Below it, the command '> db.students.insertOne({ _id: 50, name: "Mohit", age: 18, gender: "Male", course: "BBA", department: "Cyber Security", city: "Chennai", marks: 85, email: "mohit@college.edu" })' is entered. The output is '{ acknowledged: true, insertedId: 50 }'.

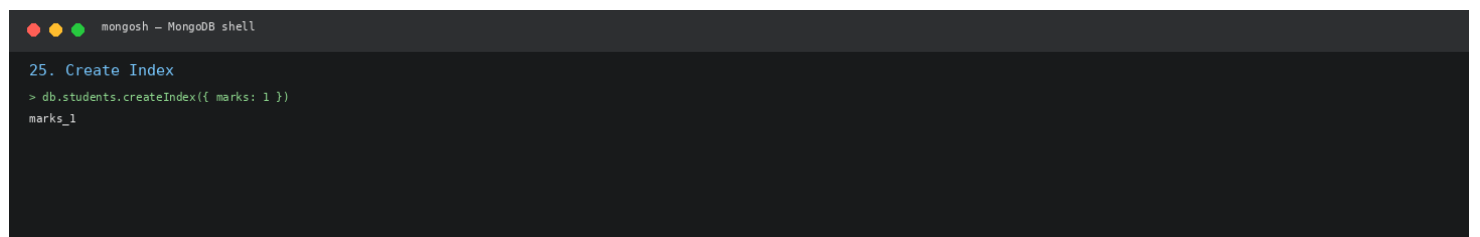
OPERATION 25: Create Index

Creates an index on marks.

MongoDB Shell Command:

```
db.students.createIndex({ marks: 1 })
```

Output Screenshot:

A screenshot of the MongoDB Shell interface. The title bar shows 'mongosh - MongoDB shell'. The main window has a dark background. The text '25. Create Index' is displayed in blue. Below it, the command '> db.students.createIndex({ marks: 1 })' is entered. The output is 'marks_1'.

OPERATION 26: View Indexes

Displays collection indexes.

MongoDB Shell Command:

```
db.students.getIndexes()
```

Output Screenshot:

A screenshot of the MongoDB Shell interface. The title bar shows 'mongosh - MongoDB shell'. The prompt is '26. View Indexes'. The command entered is '> db.students.getIndexes()'. The output shows two indexes: '_id' and 'marks_1'.

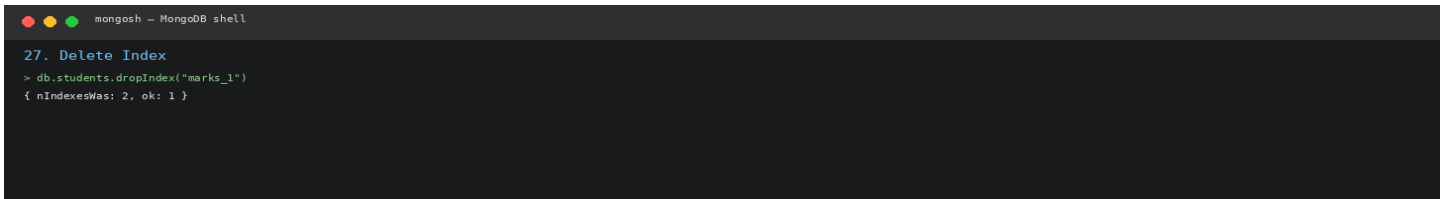
OPERATION 27: Delete Index

Deletes the marks index.

MongoDB Shell Command:

```
db.students.dropIndex("marks_1")
```

Output Screenshot:

A screenshot of the MongoDB Shell interface. The title bar shows 'mongosh - MongoDB shell'. The prompt is '27. Delete Index'. The command entered is '> db.students.dropIndex("marks_1")'. The output is '{ nIndexesWas: 2, ok: 1 }'.

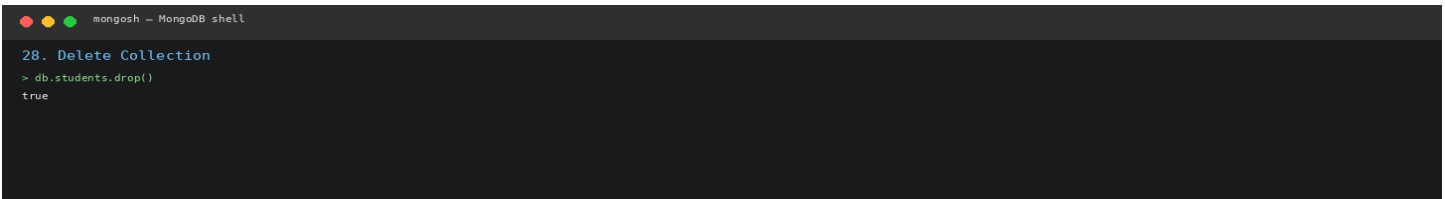
OPERATION 28: Delete Collection

Deletes the students collection as final cleanup.

MongoDB Shell Command:

```
db.students.drop()
```

Output Screenshot:

A screenshot of the MongoDB Shell interface. The title bar shows 'mongosh - MongoDB shell'. The prompt is '28. Delete Collection'. The command entered is '> db.students.drop()'. The output is 'true'.

CONCLUSION

This project demonstrated the use of MongoDB as a NoSQL document database for storing and managing student information. Exactly 50 student records were created and used for the practical operations.

The project covered database creation, collection creation, insertion, retrieval, comparison operators, logical operators, projection, sorting, limiting, counting, distinct values, updating and deleting documents.

Indexing introduced the basic concept of improving query performance. The project also showed how MongoDB commands can be combined with conditions to obtain useful student information.

Overall, this project provides a practical foundation for understanding MongoDB Shell and common NoSQL operations. The separate explanations and screenshots make each command easier to reproduce and understand.